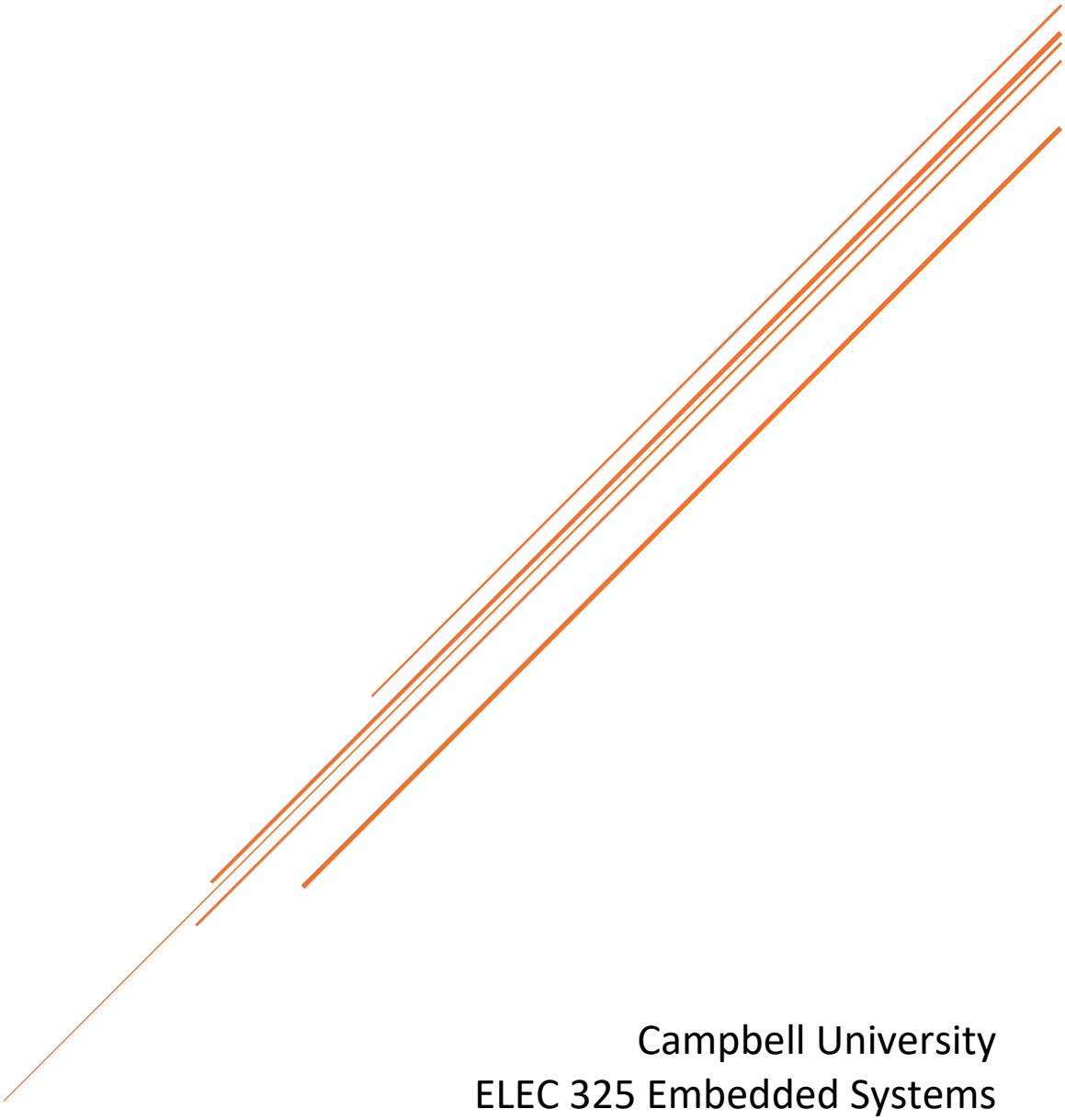
LED CONTROL SYSTEM

By Jake Stanley



Campbell University
ELEC 325 Embedded Systems
Updated: 4-28-2026
Version 1.0

© Copyright 2026 Stanley

Table of Contents

1 Abstract 3

2 References 3

3 Introduction 3

3.1 Terminology 3

4 Theory Of Operation 4

4.1 System Hardware 4

4.2 Ports 5

4.3 External LEDs 5

4.3.1 Registers 6

4.3.2 Initialization 6

4.3.3 LED operation 7

4.4 Pushbuttons 8

4.4.1 Registers 9

4.4.2 Initialization 9

4.4.3 Interrupts 10

4.5 Timer B 10

4.5.1 Registers 10

4.5.2 Clock Source 11

4.5.3 Initialization and Starting the Timer 11

4.5.4 Timer Interrupt 11

5 Design and Implementation 12

5.1 Development 12

5.2 Scheduler 12

5.3 State Machine 13

5.3.1 Off State 13

5.3.2 Speed Change State 14

5.3.3 Direction Change State 14

5.3.4 Counter State 14

© Copyright Stanley 2026

Figures

Figure 1. System Block Diagram 4
Figure 2. Circuit Diagram for External Components 4
Figure 3. Location of each button on the Launchpad [3] 8
Figure 4. Up Mode 11
Figure 5. State Machine 13

Tables

Table 1. Pin Map 5
Table 2. LED IDs 5
Table 3. LED registers 6
Table 4. Pushbutton functions 8
Table 5. Pushbutton registers 9
Table 6. Timer B Registers 10
Table 7. Speed Change button functions 14
Table 8. Direction change button functions 14
Table 9. Counter button functions 14

© Copyright Stanley 2026

1 Abstract

This report presents the design and implementation of an LED control system using the MSP430FR2355 microcontroller. The system uses pushbutton inputs, timer-based control, and a state machine to manage multiple LED operating modes. Key components such as port configuration, interrupt handling, and Timer B are used to create a responsive and non-blocking system. The report also outlines the overall system architecture and explains how each part contributes to the final behavior.

2 References

1. Texas Instruments, MSP430FR4xx and MSP430FR2xx family User's Guide, Literature Number: SLAU445I October 2014–Revised March 2019
2. Texas Instruments, "Code Composer Studio™," version 20.4.0.13__1.10.0, [Online]. Available: https://www.ti.com/tool/CCSTUDIO
3. Texas Instruments, MSP430FR2355 LaunchPad™ Development Kit (MSP-EXP430FR2355), Literature Number: SLAU680–May 2018

3 Introduction

The objective of this project is to demonstrate concepts learned in the embedded systems course. Using the MSP430FR2355 microcontroller and Code Composer Studio [2], a system was developed to control an array of LEDs through pushbutton inputs and timer based control. The system consists of 4 operating modes, each controlled by a structured state machine.

The project is considered complete when the system operates as intended and demonstrates reliable, fault-tolerant behavior.

3.1 Terminology

The System – Refers to the LED control system as a whole

Pin -- The physical pin on the Launchpad connected to a corresponding port bit on the microcontroller

GPIO – General purpose input/output

ISR – Interrupt service routine

SMCLK – Sub-main clock

BCLR – clear bit

BSET – set bit

© Copyright Stanley 2026

4 Theory Of Operation

4.1 System Hardware

An overview of the system Hardware can be seen in Figure 1

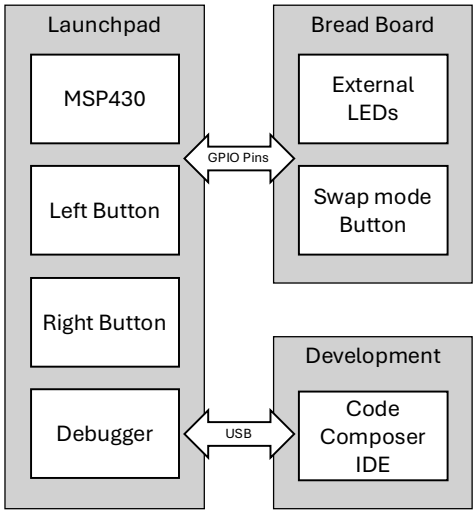


Figure 1. System Block Diagram

The system utilizes external components such as LEDs, pushbuttons, resistors, and capacitors. The circuit diagram shown in Figure 2 provides the proper configuration for these components.

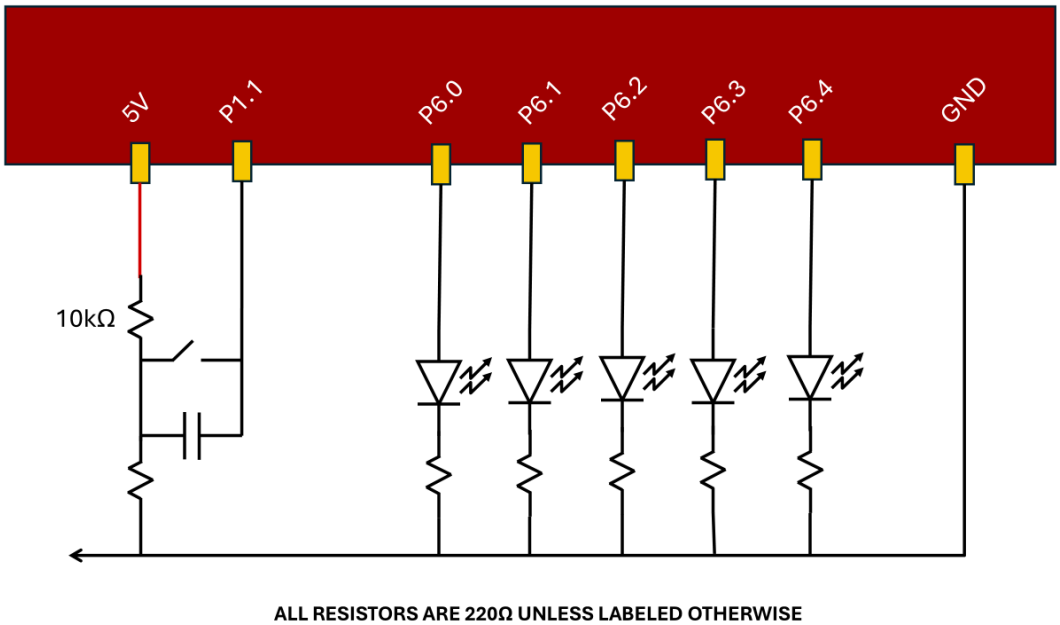


Figure 2. Circuit Diagram for External Components

© Copyright Stanley 2026

4.2 Ports

The MSP430 uses ports and individual bits within those ports to interact with external hardware [1]. Table 1 shows how each pin is used within the system.

Port	.0	.1	.2	.3	.4	.5	.6	.7
P1		PB_1						
P2				PB_2*				
P3								
P4		PB_3*						
P5								
P6	LED_1	LED_2	LED_3	LED_4	LED_5			

Table 1. Pin Map

Legend:

- PB_x: pushbutton
- LED_x: external LED
- x: ID for each component
- *Located on the launchpad [3]

4.3 External LEDs

The System uses five external LEDs all connected to port 6. Having each LED on the same port makes initialization and operation simpler because functions only have to manipulate the bits within a single port.

LED ID	Definition
LED_LOWERBOUND	Lower boundary, not a physical LED
LED_UPPERBOUND	Upper boundary, not a physical LED
LED_ALL	Controls all LEDs at once (on and off function only)
LED_1	External LED 1
LED_2	External LED 2
LED_3	External LED 3
LED_4	External LED 4
LED_5	External LED 5

Table 2. LED IDs

© Copyright Stanley 2026

4.3.1 Registers

Port 6 Registers used for LED initialization and operation can be seen in Table 3 . Additional information on registers may be found in the MSP430 User's Guide [1].

Acronym	Name	Function
P6SEL0	Port 6 Select 0	Selects pin function
P6SEL1	Port 6 Select 1	Selects pin function
P6DIR	Port 6 Direction	Defines each pin as an input or an output
P6OUT	Port 6 Output	Controls the output of each pin

Table 3. LED registers

4.3.2 Initialization

Each LED is defined in the **led_Index** array, which stores the LED ID, pin number, and current state. This allows all LEDs to be initialized using the loop shown below. The loop configures each pin as a GPIO output and ensures that all LEDs are initially turned off.

```
void led_InitLeds(void){
    uint8 i;
    for(i = 0; i < NUM_LEDS; i++){
        P6SEL0 BCLR led_Index[i].pin;
        P6SEL1 BCLR led_Index[i].pin;
        P6DIR BSET led_Index[i].pin;
        P6OUT BCLR led_Index[i].pin;
    }
}
```

© Copyright Stanley 2026

4.3.3 LED operation

The system uses three LED functions: **`led_TurnOn()`**, **`led_TurnOff()`**, and **`led_Toggle()`**. These functions control the LEDs by setting, clearing, or toggling the bits in the P6OUT register that matches the LED ID passed into the function. Since all LEDs are on Port 6, each LED can be controlled individually using its corresponding bit.

The **`led_TurnOn()`** and **`led_TurnOff()`** functions can control a single LED or all LEDs at once. When a valid LED ID is passed, the matching bit in P6OUT is changed to turn that LED on or off. If **`LED_ALL`** is passed, the function loops through all LEDs and updates each one, allowing all LEDs to be turned on or off with one call.

The **`led_Toggle()`** function works in a similar way but instead flips the current state of the selected LED. This is useful for creating blinking behavior.

Bounds checking is used to make sure only valid LED IDs are used. Values such as **`LED_UPPERBOUND`** and **`LED_LOWERBOUND`** do not represent real LEDs. Instead, they act as boundary markers used by the state logic. When these values are passed to the LED functions, nothing happens. This prevents out-of-bounds access while still allowing the system to track LED position.

© Copyright Stanley 2026

4.4 Pushbuttons

The system uses 3 pushbuttons for user input; one external button and the two buttons built into the Launchpad [3]. Figure 3 shows the location of each button in relation to the launchpad [3]. Pushbutton functions can be seen in Table 4.

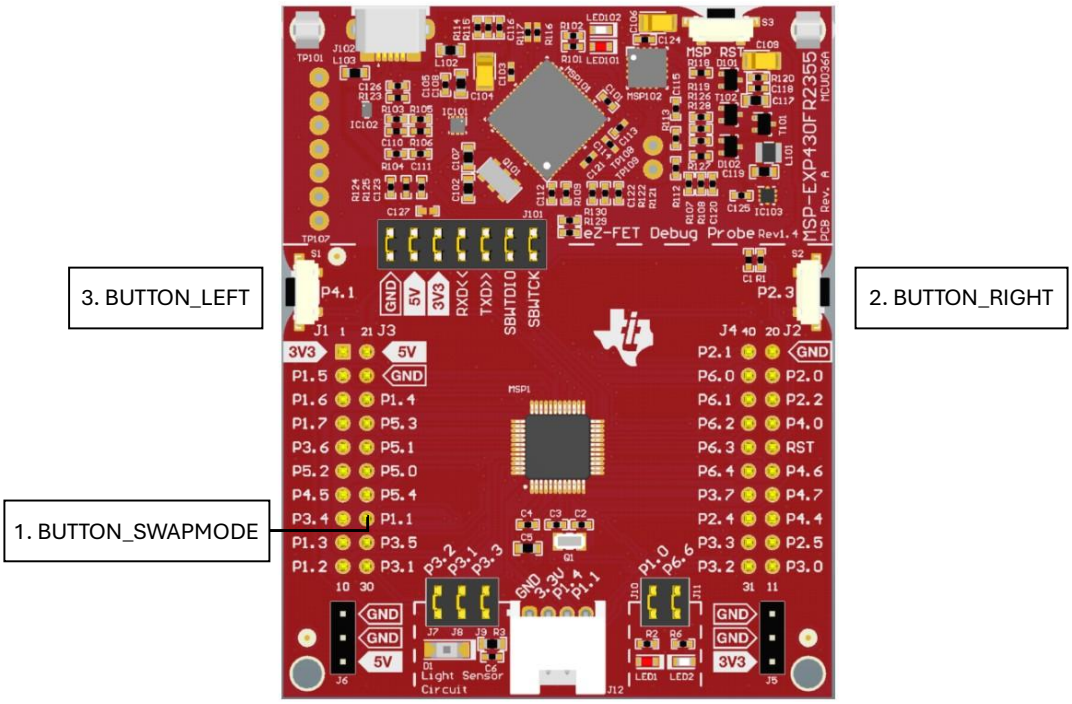


Figure 3. Location of each button on the Launchpad [3]

Button ID	Function
BUTTON_SWAPMODE	when pressed, requests for a state change to occur on the next scheduler loop
BUTTON_RIGHT	State specific function
BUTTON_LEFT	State specific function

Table 4. Pushbutton functions

© Copyright Stanley 2026

4.4.1 Registers

Port Registers used for pushbutton initialization and operation can be seen in Table 5. Additional information on registers may be found in the MSP430 User's Guide [1].

Acronym	Name	Function
PxSEL0	Port x Select 0	Selects pin function
PxSEL1	Port x Select 1	Selects pin function
PxDIR	Port x Direction	Defines each pin as an input or an output
PxREN	Port x Resistor enable	Enables use of pull up or pull down resistor
PxOUT	Port x Output	Selects pull up or pulldown resistor
PxIES	Port x Interrupt edge select	Selects the edge as low-to-high or high-to-low
PxIFG	Port x Interrupt Flag	Flags that interrupt has occurred
PxIE	Port x Interrupt enable	Enables port interrupts on selected pin

Table 5. Pushbutton registers

4.4.2 Initialization

Each button is defined in the **`btn_Index`** array, which stores the button ID, pin number, current flag value, and pointers to the port registers used by that button. Since the buttons are located on different ports with different register addresses, pointers are used to reference the correct address for each button. This allows each button to be initialized using the same loop shown below. The loop configures each pin as a GPIO input with an internal pull-up resistor and enables interrupts to detect button presses.

```
void btn_InitButtons(void){
    uint8 i;
    uint8 _pin;
    for(i = 0; i < NUM_BUTTONS; i++){
        _pin = btn_Index[i].pin;

        *(btn_Index[i].PxSEL0) BCLR _pin;
        *(btn_Index[i].PxSEL1) BCLR _pin;
        *(btn_Index[i].PxDIR) BCLR _pin;
        *(btn_Index[i].PxOUT) BSET _pin;                    //pull up resistor
        *(btn_Index[i].PxREN) BSET _pin;

        *(btn_Index[i].PxIES) BSET _pin;                   //high-to-low
        *(btn_Index[i].PxIFG) BCLR _pin;
        *(btn_Index[i].PxIE) BSET _pin;
    }
}
```

© Copyright Stanley 2026

4.4.3 Interrupts

Pushbutton input is handled using ISRs so that the controller doesn't miss any presses. When a button is pressed, an interrupt is generated on the port connected to that button. Each ISR sets the corresponding flag in the **`btn_Index`** array to show that the button has been pressed.

The swap mode button is handled slightly differently than the left and right buttons. When the swap mode ISR runs, it sets the button flag and increments the current state. If the state reaches the upper limit, it resets back to the off state. The right and left button ISRs only set their button flags, which are later handled by the current state's control logic.

At the end of each ISR, the interrupt flag is cleared using the corresponding PxIFG register. This allows the button to generate another interrupt the next time it is pressed.

4.5 Timer B

The system uses the Timer B module to provide a consistent timing reference for LED control. The timer generates interrupts at a constant interval, which increment a global counter to create a generic "tick speed." This tick counter is used by the control logic to determine when LED operations should occur by comparing it to a predefined speed constant.

The Timer B module is implemented using a general timer control file developed earlier in the course. Because of this, some parts of the timer functions go beyond what is needed for this system. Only the functionality required to generate a consistent timing reference is used and documented.

4.5.1 Registers

Registers used for Timer B initialization and control can be seen in Table 6 . Additional information on registers may be found in the MSP430 User's Guide [1].

Acronym	Name	Function
TBxCTL	Timer B Control	Controls timer operation, including clock source, mode, and prescaler settings.
TBxCCTLn	Timer B Capture/Compare Control	Configures capture/compare mode and enables interrupts for CCRn.
TBxCCRn	Timer B Capture/Compare	stores the data used for comparison with timer count to trigger an interrupt.
TBxEX0	Timer B Expansion	Selects additional divider for input clock.

Table 6. Timer B Registers

© Copyright Stanley 2026

4.5.2 Clock Source

The system uses the SMCLK as the clock source for the timer. The SMCLK runs at a relatively high frequency of about 1 MHz. The higher clock speed allows for better resolution, which makes it useful for applications that require more precise timing, like the LED control used in the system.

4.5.3 Initialization and Starting the Timer

Each Timer B module uses the same memory offsets, which means they can be configured by pointing to the base address of the timer, rather than pointing to each individual register in that timer. Each Timer B module is assigned a pointer in the **`tmr_TimerBIndex`** that's points to its base address in memory. The pointers can then be used to access and configure each timer's registers through the same structure. However, the system only uses one of these timer modules.

The timer is started using the **`tmr_StartTimer()`** function, which selects which module, clock source, and operating mode to use. In this system, Timer B0 is used with SMCLK in up mode. Inside the function, a pointer to the selected module is used to configure its registers. The timer is reset, and the selected clock source and mode are applied through the control register. Additional divider settings are applied and the compare value stored in **`TBxCCR0`** is set to generate interrupts at about 10ms intervals.

4.5.3.1 Up Mode

When the timer is in up mode, it starts at zero, counts up to the compare value, and then resets back to zero as shown in Figure 4. Every time the timer reaches the compare value it triggers an interrupt.

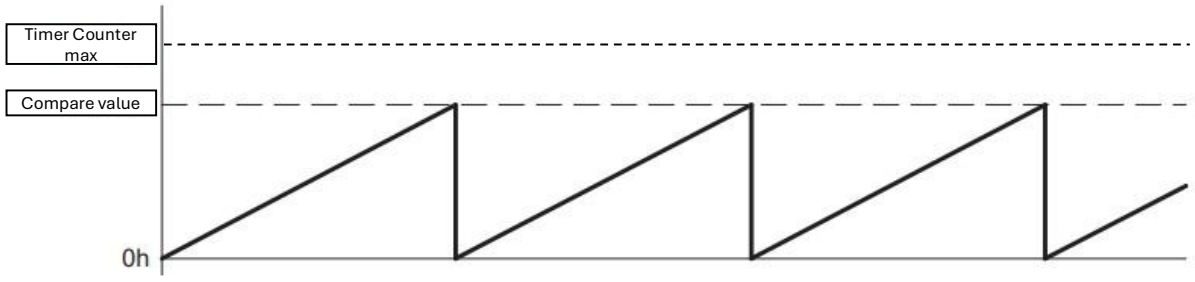


Figure 4. Up Mode

4.5.4 Timer Interrupt

The timer ISR increments the global tick counter and then clears the CCR0 and timer interrupt flags to allow the next interrupt to occur.

© Copyright Stanley 2026

5 Design and Implementation

5.1 Development

The development process began by defining the system requirements. Based on these requirements, four operating states, including an off state, were implemented using a state machine design. The system was developed incrementally, starting with basic LED control, followed by implementing interrupt based pushbutton input. A timer was then configured to allow for more advanced control of the LED blink rate. Once the input and output functionality was working properly, the state machine logic was added to control LED behavior. Each part of the system was tested individually before being integrated into the full system, and debugging was performed to fix issues related to timing, state transitions, and out-of-bounds behavior in the LED control logic.

5.2 Scheduler

The system uses the scheduler to control overall program execution. Its main job is to detect when a state change has occurred and then set the state machine to run the logic for the selected state.

The scheduler begins by calling the general initialization function. It then enters a continuous loop where it gets the current state and passes that state into the state machine. The state machine continuously runs that state logic until the swap mode button flag is set. Once that flag is set, the scheduler exits the inner loop, gets the new state, clears the flag, and repeats the process. This keeps each state running until a state change is requested, while still allowing the button interrupts and timer interrupts to update the system in the background.

© Copyright Stanley 2026

5.3 State Machine

The system uses a state machine to control the different LED operating modes. Each state contains its own logic for how the LEDs work and what each pushbutton does. The swap mode button is used to move between the different states as shown in Figure 5.

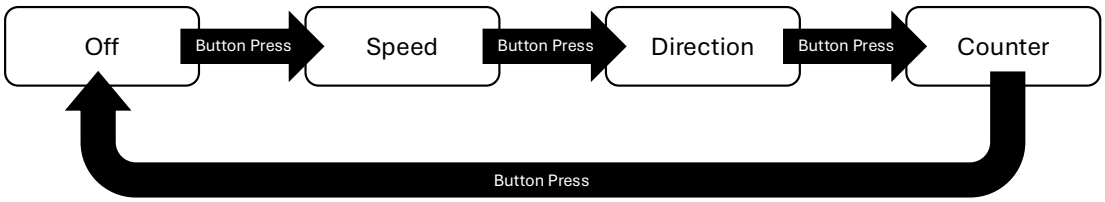


Figure 5. State Machine

The state machine determines which logic to run based on the **`sm_State`** global variable passed in by the scheduler. This is different from the **`sm_CurrState`** variable which is used to track state changes as they occur and store that information for when it is needed. This separation allows the current state to be updated without immediately interrupting the logic that is already running.

The **`sm_PrevState`** variable is used to detect when a state change has occurred. By comparing the current state to the previous state, the system can determine when a state change has just occurred and apply any initial conditions for a new state. This ensures that setup conditions, like resetting timers, setting blink speed, or resetting led position are only performed when the state has just been entered.

5.3.1 Off State

The off state turns all LEDs off on the first pass and then does nothing. The timer count is periodically reset to prevent the **`tmr_Ticks`** variable from overflowing. The left and right buttons have no function in this state.

© Copyright Stanley 2026

5.3.2 Speed Change State

The speed change state causes the LEDs to sweep across in sequence. The system uses the timer tick counter to control when the current LED should be toggled by comparing it to the **`sm_BlinkSpeed`** variable, creating a continuous sweeping pattern. After each update, the LED index increments and wraps back to the first LED when the end is reached. Pressing the buttons increases or decreases the **`sm_BlinkSpeed`** variable which changes the speed of the sweep.

Button Name	Function
BUTTON_RIGHT	Slows the LEDs down
BUTTON_LEFT	Speeds the LEDs up

Table 7. Speed Change button functions

5.3.3 Direction Change State

The direction change state causes the LEDs to move in a single direction across the array. The system uses the timer tick counter to determine when to update the current LED position, creating a stepping pattern. Pressing either button will change the direction of movement, allowing the LEDs to move left or right across the array.

Button Name	Function
BUTTON_RIGHT	Move the LED right
BUTTON_LEFT	Move the LED left

Table 8. Direction change button functions

5.3.4 Counter State

The counter state creates a fill effect using the LEDs. The system uses the timer tick counter to control when the LEDs update, allowing the animation to occur at a consistent rate. A current position and target value are used to determine how many LEDs should be active, and the system updates the LEDs progressively to match the target.

When adding an LED, the target will move down the array and trigger an animation to occur where an LED appears and slides into the target LED. When subtracting an LED, the target will move up the array and trigger an animation to occur where the current LED will slide off the array. Pressing the buttons increases or decreases the target value, which controls the number of active LEDs displayed.

Button Name	Function
BUTTON_RIGHT	Adds an LED
BUTTON_LEFT	Subtracts an LED

Table 9. Counter button functions

© Copyright Stanley 2026